$ COE203 - Project 2

Trendyol laptop scraper

DATE

2025-12-10

SECTION

COE203 - Advanced Programming with Python (1)

PRESENTER

ABED ALAZEZ AL MOHAMMAD
2389116389

GROUP

SWADZ

$ Our Development Team

MAHMOUD KONIALI

2209115583

LEAD DEVELOPER

ABED ALAZEZ AL MOHAMMAD

2389116389

PRESENTER

ABDÜLAZIZ ASLAN

2389116462

TESTING & QUALITY ASSURANCE

MOHAMMED HILALOĞLU

2389116562

PROJECT SUPPORT

SINA ABAZID

2389115511

DOCUMENTATION & RESEARCH

TRENDYOL LAPTOPS SCRAPER

Project Goal

To create a professional, production-ready web scraper using Scrapy and MongoDB, strictly adhering to advanced OOP principles and Full Type Checking.

Core Technologies

Scrapy (Web Scraping), **MongoDB** (Persistence), **Pydantic** (Data Modeling), **Pytest** (Testing), and Docker (Deployment).

ADVANCED OOP & ARCHITECTURAL EXCELLENCE

To achieve maximum points for architecture, a strict separation of concerns was enforced using advanced OOP principles.

Repository Pattern Decouples Logic from Persistence

Pydantic Data Models

The **LaptopItem** uses Pydantic for data transfer objects (DTOs), ensuring strict data validation and type safety across the entire pipeline.

Abstract Base Classes (ABC)

The **LaptopRepositoryABC** defines the contract for all database operations, decoupling the core logic from the specific database implementation (MongoDB).

Full Type Checking

All modules, including the spider, pipeline, and repository, utilize **Python Type Hints**, which significantly improves code maintainability and robustness.

ROBUST SCRAPING AND DATA PIPELINE

The solution is built on Scrapy's powerful framework, integrating a custom pipeline for data processing and persistence.

1. Scrapy Spider

- Handles complex pagination (`?pi=N`).
- Robust price parsing for European formats.
- Respects `robots.txt` and uses retry logic.

2. Pipeline Validation

- Uses the Pydantic `LaptopItem` for strict data validation.
- Drops malformed or incomplete items early.
- Ensures data quality before persistence.

3. MongoDB Repository

- Stores data using Beanie (ODM).
- Unique Index on URL prevents duplicate entries.
- Decoupled persistence layer via Repository Pattern.

CODE QUALITY, TESTING, AND DEPLOYMENT

Comprehensive Testing and Docker Enable Production Readiness

Pytest Suite

A comprehensive test suite ensures reliability and correctness:

- ✓ **Unit Tests:** Spider's parsing logic with mocked HTML responses.
- ✓ **Integration Tests:** MongoDB repository's connection and duplicate-prevention features.
- ✓ Tests validate the entire data pipeline end-to-end.

Pylint Score

High code quality and adherence to PEP8 standards:

8.24/10

Confirms clean code, minimal complexity, and best practices throughout the codebase.

Docker Deployment

Production-ready containerization:

- ✓ **Dockerfile:** Lightweight Python image with dependencies.
- ✓ **docker-compose.yml:** Orchestrates Scraper, MongoDB, and optional FastAPI API.
- ✓ One-command setup for the entire environment.

BONUS DELIVERABLE: FASTAPI DATA API

Data is Accessible via a Simple RESTful Endpoint

Technology

FastAPI, a modern, high-performance Python web framework, provides a clean and intuitive API interface for accessing the scraped data.

Functionality

Provides endpoints to retrieve all scraped laptops (**/laptops**) and the total count (**/laptops/count**).

Architecture

The API utilizes the same **MongoLaptopRepository** instance, demonstrating the power of the Repository Pattern in sharing data access logic across different application layers.

Interactive Docs

FastAPI automatically generates interactive API documentation at **/docs** (Swagger UI), making it easy to test and explore endpoints.

Run Command

```
python run_scraper.py
```